

ClusterLuck: Solver Description for Rooted Maximum Agreement Forest

Rishank Goyal ✉

The Adelaide University, Australia

Srinibas Swain ✉

The Adelaide University, Australia

Abstract

ClusterLuck is a C++17 solver for the PACE 2026 exact track for rooted Maximum Agreement Forest on rooted binary trees. The submitted program is a single statically linked executable. It dispatches two-tree instances to a bundled RS [1]/HiGHS (an ILP solver) route and instances with at least three trees to a many-tree core using validated component construction, common-cluster recursion, SAT decision certificates, and exhaustive small-savings packing. Every printed forest is checked by an independent agreement-forest validator.

2012 ACM Subject Classification Theory of computation → Design and analysis of algorithms; Applied computing → Bioinformatics

Keywords and phrases maximum agreement forest, rooted phylogenetic trees, SAT, cluster decomposition

Category PACE 2026 Exact Track Solver Description

Supplementary Material <https://github.com/alienblack/ClusterLuck>

1 Model and Validation

The input trees $T_1, \dots, T_t$ share a taxon set X , $|X| = n$. An answer is a partition $\mathcal{F} = \{C_1, \dots, C_k\}$ of X . It is a rooted agreement forest if each restriction $T_j|_{C_i}$ has the same rooted topology for all trees j , and if the minimal connecting subtrees of different components are edge-disjoint in every input tree. The objective is to minimise k .

The validator used by the solver implements exactly these two conditions. For each block it computes a canonical rooted restriction signature in every input tree and compares the signatures. It then records the edges used by the block's minimal connecting subtree in every input tree and rejects the candidate if any edge is used by two blocks. It also checks that every taxon appears exactly once.

► **Lemma 1.** *Every forest accepted by the validator is a feasible rooted agreement forest.*

Proof. The leaf checks show that the blocks partition X . Equality of canonical restriction signatures gives the same rooted topology for each block in all input trees. The recorded edge sets being pairwise disjoint in each input tree gives the edge-disjointness requirement. These are precisely the rooted agreement-forest conditions. ◀

2 Many-Tree Core

For number of input trees, $t \geq 3$, ClusterLuck first searches for a small validated forest by component-pool heuristics: common subtree and cherry reductions, chain-derived components, anchored greedy merging, high-divergence packing, pair augmentation, local exchange, and bounded repacking. These routines only provide upper bounds. They do not certify optimality unless one of the exact decision routines below proves that a smaller forest is impossible.

The solver also uses common-cluster recursion, following the standard role of cluster partitioning in MAF computation [2] and marker semantics used in cluster reductions [3]. If a leaf set C is a clade in every input tree, the instance is split into the inside of C and the outside of C , with marker variants on both sides. Let B, T be the optimum component counts for the unmarked bottom and top subinstances, and let BM, TM be the marker variants. If $BM = B$ and $TM = T$, the boundary component can be joined and the combined count is $B + T - 1$; otherwise the combined count is $B + T$. The recombined forest is again checked by the global validator.

3 Exact Certificates

After finding a validated incumbent with k components, the certified many-tree route attempts to prove that no forest with at most $k - 1$ components exists. If this succeeds, the incumbent is optimal.

Direct-pair SAT.

For smaller instances the solver creates a Boolean variable x_{ab} for each leaf pair, meaning that leaves a and b are in the same component. Transitivity clauses make these variables an equivalence relation. Clauses forbid every triple of leaves whose rooted topology is not common to all input trees from lying in one component, and edge clauses forbid two components from sharing an input-tree edge. Representative variables with a sequential counter enforce at most K components. Any K -component agreement forest would satisfy this formula; therefore UNSAT proves that target K is impossible.

Reference-cut SAT.

The solver also encodes cuts in the first input tree. A forest with at most K components corresponds to at most $K - 1$ reference cuts. For every conflicting triple, at least one reference edge on the relevant paths must be cut. SAT models are decoded to partitions and validated; invalid decoded partitions are either repaired or blocked by lazy clauses. If this lazy search exhausts and returns UNSAT, no valid K -component agreement forest exists.

Small-savings packing.

For targets close to the singleton forest, write $s = n - K$. The solver enumerates all valid components that can contribute to these savings, stores their leaf and edge masks, and solves the resulting exact packing problem. A component C contributes $|C| - 1$ savings. Thus a valid disjoint packing of savings at least s is equivalent to a forest with at most K components. When this exhaustive search finishes without such a packing, target K is proved impossible.

► **Theorem 2.** *If the many-tree core prints a validated forest with k components and one of the exact routines proves target $k - 1$ impossible, then the output is optimal.*

Proof. Validation gives a feasible forest, so $OPT \leq k$. The exact certificate proves that no forest with at most $k - 1$ components exists, so $OPT \geq k$. Hence $OPT = k$. ◀

4 Two-Tree Route and Implementation

For $t = 2$ and $n \leq 500$, the wrapper dispatches to a bundled two-tree route based on the RS/Whidden code path [1] and an h56 component-pool repair layer. It builds an incumbent

forest, collects components from incumbents and milestone forests, enriches the pool by local edits and region generators, and solves set-packing subproblems using the statically linked HiGHS library. The final forest is repaired and validated before output. For $t = 2, n > 500$, the solver intentionally times out instead of returning an untrusted large answer.

The submitted executable is `solver_submit`. It vendors CaDiCaL for SAT and HiGHS for integer programming, stores trees in compact arrays with LCA support, and writes one Newick component per output line.

This build enables the full candidate-generation pipeline around the exact core: high-divergence component generation, exact-exchange local replacement, pair augmentation, incumbent repacking, and additional time-budgeted pool searches. These phases only enlarge the set of candidate components and replacement forests considered by the solver; every candidate that is kept is still checked by the same agreement-forest validator.

References

- 1 C. Whidden, R. G. Beiko, and N. Zeh. Fixed-parameter algorithms for maximum agreement forests. *SIAM Journal on Computing*, 42(4):1431–1466, 2013.
- 2 Z. Li and N. Zeh. Computing maximum agreement forests without cluster partitioning is folly. In *25th Annual European Symposium on Algorithms (ESA 2017)*, LIPIcs 87, pages 56:1–56:14, 2017.
- 3 S. Kelk. A pedagogical implementation of the cluster reduction. <https://skelk.sdf-eu.org/cluster/>, accessed 2026-07-09.
- 4 A. Biere, K. Fazekas, M. Fleury, and M. Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. In *Proc. SAT Competition 2020*, 2020.
- 5 Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018.